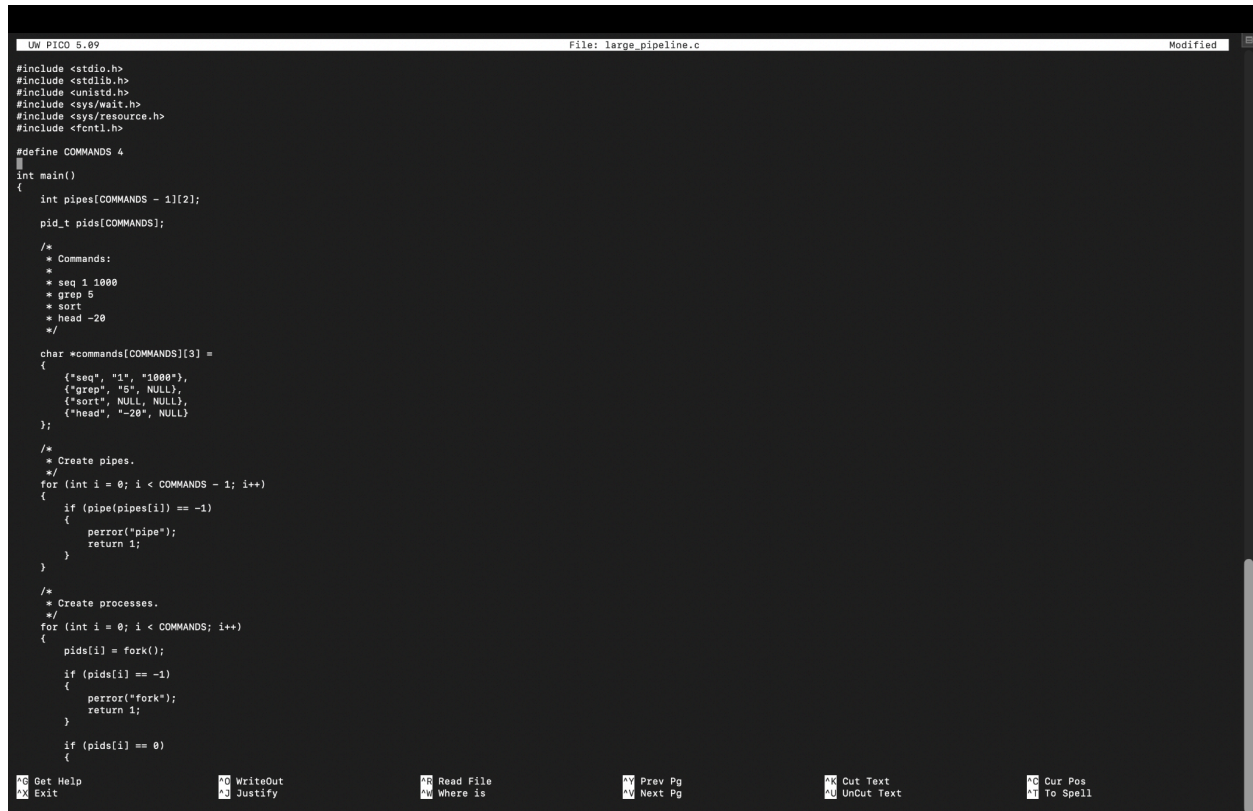


WEEK12-SKILL

Task1:

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance



```
UW PICO 5.89 File: large_pipeline.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <fcntl.h>

#define COMMANDS 4

int main()
{
    int pipes[COMMANDS - 1][2];
    pid_t pids[COMMANDS];

    /*
     * Commands:
     *
     * seq 1 1000
     * grep 5
     * sort
     * head -20
     */

    char *commands[COMMANDS][3] =
    {
        {"seq", "1", "1000"},
        {"grep", "5", NULL},
        {"sort", NULL, NULL},
        {"head", "-20", NULL}
    };

    /*
     * Create pipes.
     */
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return i;
        }
    }

    /*
     * Create processes.
     */
    for (int i = 0; i < COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
            perror("fork");
            return i;
        }

        if (pids[i] == 0)
        {
            if (i > 0)
            {
                if (fcntl(pipes[i-1][1], F_SETFD, FD_CLOEXEC) == -1)
                {
                    perror("fcntl");
                    return i;
                }
                if (read(pipes[i-1][1], &buf, sizeof(buf)) == -1)
                {
                    perror("read");
                    return i;
                }
                if (write(pipes[i][0], &buf, sizeof(buf)) == -1)
                {
                    perror("write");
                    return i;
                }
            }
            if (i == 0)
            {
                if (execvp(commands[i][0], commands[i]) == -1)
                {
                    perror("execvp");
                    return i;
                }
            }
        }
    }

    if (waitpid(pids[0], &status, 0) == -1)
    {
        perror("waitpid");
        return 0;
    }

    if (WIFEXITED(status))
    {
        if (WEXITSTATUS(status) != 0)
        {
            return 0;
        }
    }

    return 0;
}
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

```
UW PICO 5.09 File: large_pipeline.c Modified
};

/*
 * Create pipes.
 */
for (int i = 0; i < COMMANDS - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");
        return 1;
    }
}

/*
 * Create processes.
 */
for (int i = 0; i < COMMANDS; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        return 1;
    }

    if (pids[i] == 0)
    {
        /*
         * If this is not the first command,
         * connect stdin to previous pipe.
         */
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /*
         * If this is not the last command,
         * connect stdout to next pipe.
         */
        if (i < COMMANDS - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /*
         * Close all pipe descriptors.
         */
        for (int j = 0; j < COMMANDS - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        /*
         * Last command writes to output.txt.
         */
    }
}
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

```
UW PICO 5.09 File: large_pipeline.c Modified
    /*
     * If this is not the first command,
     * connect stdin to previous pipe.
     */
    if (i > 0)
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }

    /*
     * If this is not the last command,
     * connect stdout to next pipe.
     */
    if (i < COMMANDS - 1)
    {
        dup2(pipes[i][1], STDOUT_FILENO);
    }

    /*
     * Close all pipe descriptors.
     */
    for (int j = 0; j < COMMANDS - 1; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    /*
     * Last command writes to output.txt.
     */
    if (i == COMMANDS - 1)
    {
        int fd = open(
            "pipeline_output.txt",
            O_WRONLY | O_CREAT | O_TRUNC,
            0644
        );

        if (fd == -1)
        {
            perror("output file");
            exit(1);
        }

        dup2(fd, STDOUT_FILENO);
        close(fd);

        /*
         * Execute command.
         */
        execvp(commands[i][0], commands[i]);

        perror("execvp");
        exit(127);
    }
}

/*
```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text ⌘ Cur Pos
⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ To Spell

```
UW PICO 5.09 File: large_pipeline.c Modified
{
    int fd = open(
        "pipeline_output.txt",
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );
    if (fd == -1)
    {
        perror("output file");
        exit(1);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
}
/*
 * Execute command.
 */
execvp(commands[i][0], commands[i]);
perror("execvp");
exit(127);
}
}
/*
 * Parent closes all pipe descriptors.
 */
for (int i = 0; i < COMMANDS - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}
/*
 * Wait for every process.
 */
int failures = 0;
for (int i = 0; i < COMMANDS; i++)
{
    int status;
    if (waitpid(pids[i], &status, 0) == -1)
    {
        perror("waitpid");
        failures++;
        continue;
    }
    if (WIFEXITED(status))
    {
        int exit_status = WEXITSTATUS(status);
        printf("Process %d exited with status %d\n",
            pids[i],
            exit_status);
    }
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: large_pipeline.c Modified
/*
 * Parent closes all pipe descriptors.
 */
for (int i = 0; i < COMMANDS - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}
/*
 * Wait for every process.
 */
int failures = 0;
for (int i = 0; i < COMMANDS; i++)
{
    int status;
    if (waitpid(pids[i], &status, 0) == -1)
    {
        perror("waitpid");
        failures++;
        continue;
    }
    if (WIFEXITED(status))
    {
        int exit_status = WEXITSTATUS(status);
        printf("Process %d exited with status %d\n",
            pids[i],
            exit_status);
        if (exit_status != 0)
        {
            failures++;
        }
    }
    else if (WIFSIGNALED(status))
    {
        printf("Process %d terminated by signal %d\n",
            pids[i],
            WTERMSIG(status));
        failures++;
    }
}
/*
 * Resource usage of the current process
 * and its children.
 */
struct rusage usage;
if (getrusage(RUSAGE_CHILDREN, &usage) == 0)
{
    printf("\nResource Usage\n");
    printf("-----\n");
    printf("User CPU time : %ld.%ld seconds\n",
        usage.ru_utime.tv_sec,
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: large_pipeline.c Modified

    if (exit_status != 0)
    {
        failures++;
    }
}
else if (WIFSIGNALED(status))
{
    printf("Process %d terminated by signal %d\n",
           pidg[i],
           WTERMSIG(status));
    failures++;
}
}

/*
 * Resource usage of the current process
 * and its children.
 */
struct rusage usage;
if (getrusage(RUSAGE_CHILDREN, &usage) == 0)
{
    printf("\nResource Usage\n");
    printf("-----\n");

    printf("User CPU time : %ld.%ld seconds\n",
           usage.ru_utime.tv_sec,
           usage.ru_utime.tv_usec);

    printf("System CPU time : %ld.%ld seconds\n",
           usage.ru_stime.tv_sec,
           usage.ru_stime.tv_usec);

    printf("Maximum memory : %ld KB\n",
           usage.ru_maxrss);
}

printf("\nPipeline Analysis\n");
printf("-----\n");

if (failures == 0)
{
    printf("Pipeline completed successfully.\n");
}
else
{
    printf("Pipeline had %d failure(s).\n",
           failures);
}

return failures == 0 ? 0 : 1;
}

Get Help      WriteOut      Read File     Prev Pg       Cut Text      Cur Pos
Exit          Justify        Where is      Next Pg       UnCut Text    To Spell
```

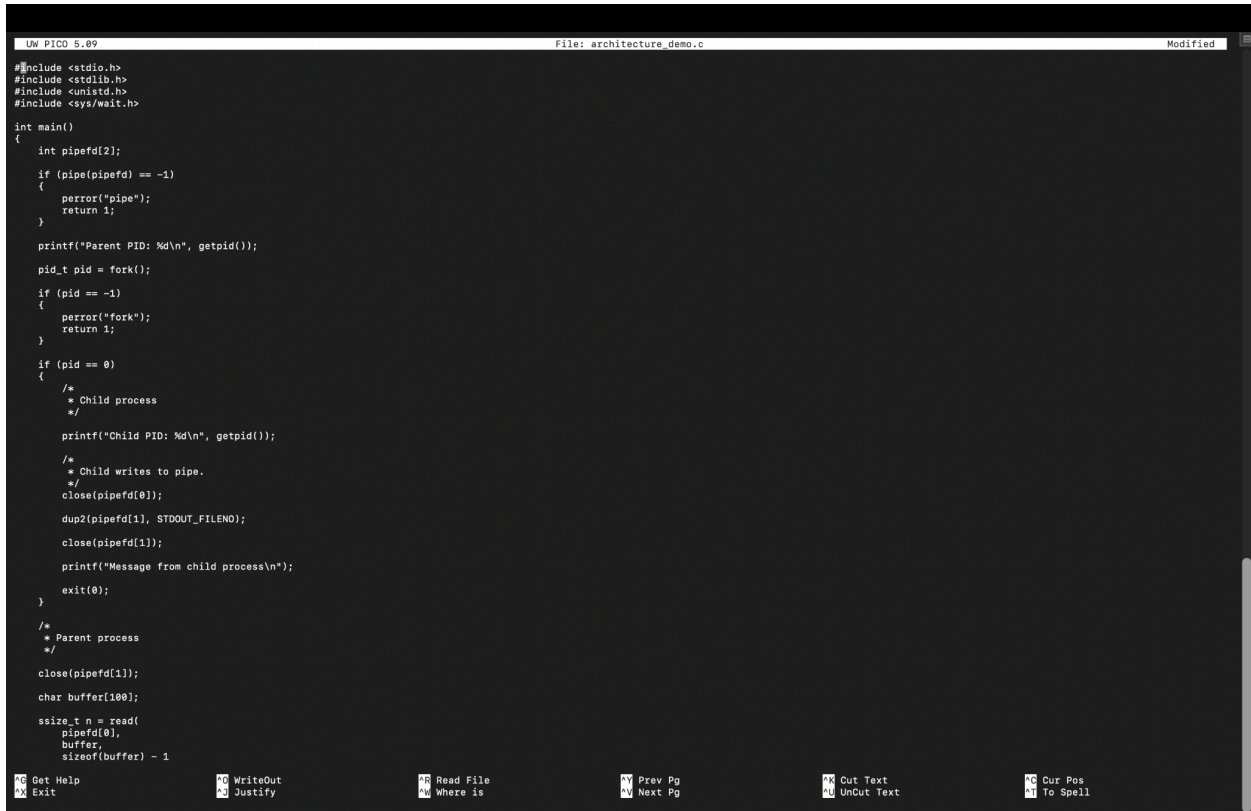
```
[vishalkarnati@VISHALS-MacBook-Air week12skill % nano large_pipeline.c
[vishalkarnati@VISHALS-MacBook-Air week12skill % gcc large_pipeline.c -o large_pipeline
[vishalkarnati@VISHALS-MacBook-Air week12skill % ./large_pipeline
usage: seq [-w] [-f format] [-s string] [-t string] [first [incr]] last
Process 1749 exited with status 1
Process 1750 exited with status 1
Process 1751 exited with status 0
Process 1752 exited with status 0

Resource Usage
-----
User CPU time : 0.003920 seconds
System CPU time : 0.012275 seconds
Maximum memory : 1392640 KB

Pipeline Analysis
-----
Pipeline had 2 failure(s).
[vishalkarnati@VISHALS-MacBook-Air week12skill % ]
```

Task2:

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation



The screenshot shows a code editor window titled "UW PICO 5.09" with a file named "architecture_demo.c". The code is a C program that demonstrates process communication using pipes. It includes standard headers like `<stdio.h>`, `<stdlib.h>`, `<unistd.h>`, and `<sys/wait.h>`. The `main` function starts by declaring a pipe `pipefd[2]`. It then checks if the pipe creation failed (`pipe(pipefd) == -1`). If successful, it prints the parent PID and forks a child process. The child process (`pid == 0`) writes a message to the pipe, closes the write end, and exits. The parent process (`pid == -1`) closes the write end, reads the message from the pipe, and prints it. The program uses `printf` for output and `exit` to terminate.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Parent PID: %d\n", getpid());
    pid_t pid = fork();
    if (pid == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Child process
         */
        printf("Child PID: %d\n", getpid());
        /*
         * Child writes to pipe.
         */
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        printf("Message from child process\n");
        exit(0);
    }

    /*
     * Parent process
     */
    close(pipefd[1]);
    char buffer[100];
    ssize_t n = read(
        pipefd[0],
        buffer,
        sizeof(buffer) - 1
    );
}
```

At the bottom of the editor, there is a toolbar with various icons and labels for actions like "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Cur Pos", and "To Spell".

```
UW PICO 5.09 File: architecture_demo.c Modified

printf("Child PID: %d\n", getpid());

/*
 * Child writes to pipe.
 */
close(pipefd[0]);

dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[1]);

printf("Message from child process\n");
exit(0);
}

/*
 * Parent process
 */
close(pipefd[1]);
char buffer[100];
ssize_t n = read(
    pipefd[0],
    buffer,
    sizeof(buffer) - 1
);

if (n > 0)
{
    buffer[n] = '\0';
    printf("Parent received: %s",
        buffer);
}

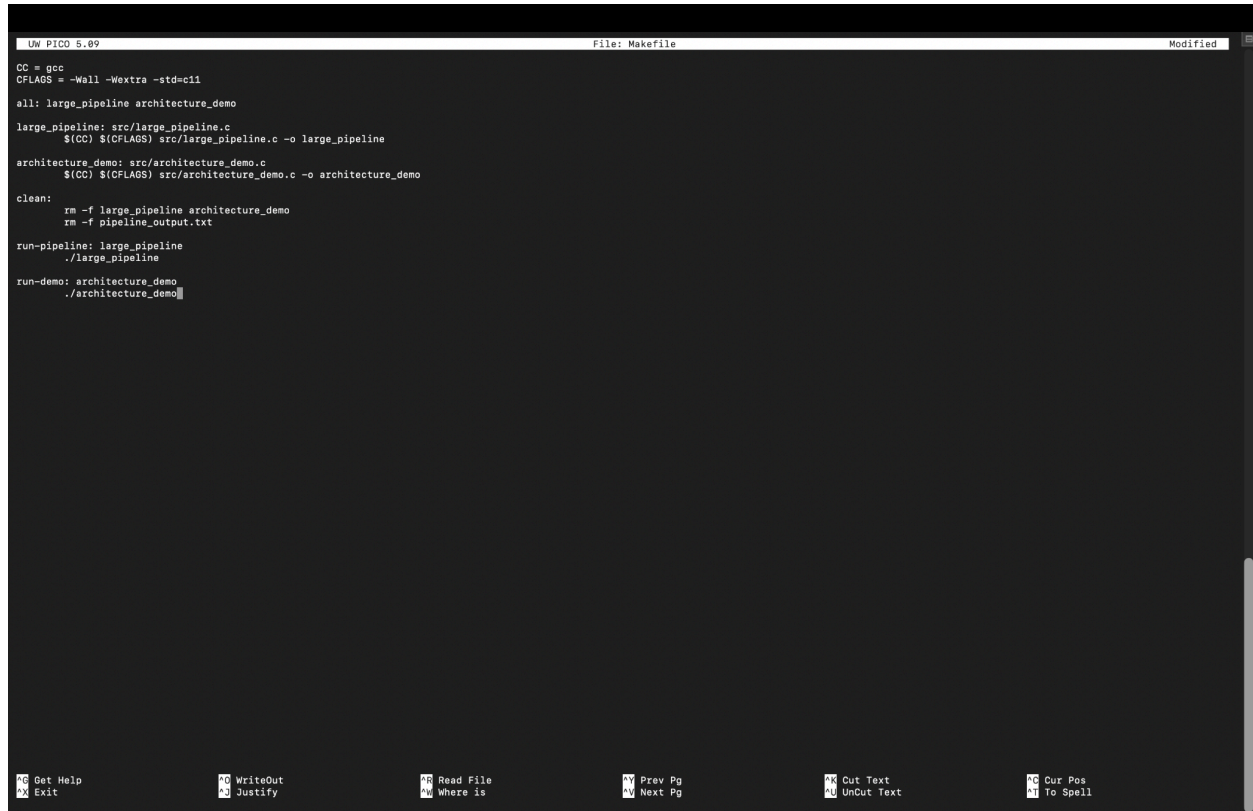
close(pipefd[0]);
waitpid(pid, NULL, 0);
printf("Parent completed.\n");
return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week12$ kill % nano architecture_demo.c
vishalkarnati@VISHALS-MacBook-Air week12$ kill % gcc architecture_demo.c -o architecture_demo
vishalkarnati@VISHALS-MacBook-Air week12$ kill % ./architecture_demo
Parent PID: 1769
Child PID: 1770
Parent received: Message from child process
Parent completed.
vishalkarnati@VISHALS-MacBook-Air week12$ kill %
```

Task3:

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables



The screenshot shows a Vim editor window with a dark background. The title bar at the top indicates the file is 'Makefile' and it is 'Modified'. The main text area contains a Makefile with the following content:

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11

all: large_pipeline architecture_demo

large_pipeline: src/large_pipeline.c
    $(CC) $(CFLAGS) src/large_pipeline.c -o large_pipeline

architecture_demo: src/architecture_demo.c
    $(CC) $(CFLAGS) src/architecture_demo.c -o architecture_demo

clean:
    rm -f large_pipeline architecture_demo
    rm -f pipeline_output.txt

run-pipeline: large_pipeline
    ./large_pipeline

run-demo: architecture_demo
    ./architecture_demo
```

At the bottom of the window, there is a status bar with several icons and text labels: 'Get Help', 'Exit', 'WriteOut', 'Justify', 'Read File', 'Where is', 'Prev Pg', 'Next Pg', 'Cut Text', 'UnCut Text', 'Cur Pos', and 'To Spell'.

```

vishalkarnati@VISHALS-MacBook-Air week12skill % mkdir -p src tests docs
vishalkarnati@VISHALS-MacBook-Air week12skill % mv large_pipeline.c src/
vishalkarnati@VISHALS-MacBook-Air week12skill % mv architecture_demo.c src/
vishalkarnati@VISHALS-MacBook-Air week12skill % find . -maxdepth 2 -type f
./large_pipeline
./architecture_demo
./pipeline_output.txt
./src/architecture_demo.c
./src/large_pipeline.c
vishalkarnati@VISHALS-MacBook-Air week12skill % nano Makefile
vishalkarnati@VISHALS-MacBook-Air week12skill % gcc large_pipeline.c -o large_pipeline
clang: error: no such file or directory: 'large_pipeline.c'
clang: error: no input files
vishalkarnati@VISHALS-MacBook-Air week12skill % make
make: Nothing to be done for 'all'.
vishalkarnati@VISHALS-MacBook-Air week12skill % gcc src/large_pipeline.c -o large_pipeline
vishalkarnati@VISHALS-MacBook-Air week12skill % make clean
rm -f large_pipeline architecture_demo
rm -f pipeline_output.txt
vishalkarnati@VISHALS-MacBook-Air week12skill % make
gcc -Wall -Wextra -std=c11 src/large_pipeline.c -o large_pipeline
gcc -Wall -Wextra -std=c11 src/architecture_demo.c -o architecture_demo
vishalkarnati@VISHALS-MacBook-Air week12skill % find . -maxdepth 2 -type f
./Makefile
./large_pipeline
./architecture_demo
./src/architecture_demo.c
./src/large_pipeline.c
vishalkarnati@VISHALS-MacBook-Air week12skill % make run-pipeline
./large_pipeline
usage: seq [-w] [-f format] [-s string] [-t string] [first [incr]] last
Process 1829 exited with status 1
Process 1830 exited with status 1
Process 1831 exited with status 0
Process 1832 exited with status 0

Resource Usage
-----
User CPU time : 0.005969 seconds
System CPU time : 0.016168 seconds
Maximum memory : 1392640 KB

Pipeline Analysis
-----
Pipeline had 2 failure(s).
make: *** [run-pipeline] Error 1
vishalkarnati@VISHALS-MacBook-Air week12skill % cat pipeline_output.txt
vishalkarnati@VISHALS-MacBook-Air week12skill % █

```

Task4:

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
vishalkarnati@VISHALS-MacBook-Air week12skill % ls
architecture_demo  docs          large_pipeline  Makefile        pipeline_output.txt  src          tests
architecture_demo.c  large_pipeline.c
vishalkarnati@VISHALS-MacBook-Air week12skill % ./large_pipeline
usage: seq [-w] [-f format] [-s string] [-t string] [first [incr]] last
Process 1848 exited with status 1
Process 1849 exited with status 1
Process 1850 exited with status 0
Process 1851 exited with status 0

Resource Usage
-----
User CPU time : 0.005134 seconds
System CPU time : 0.007768 seconds
Maximum memory : 1372640 KB

Pipeline Analysis
-----
Pipeline had 2 failure(s).
vishalkarnati@VISHALS-MacBook-Air week12skill % cat pipeline_output.txt
vishalkarnati@VISHALS-MacBook-Air week12skill % ./architecture_demo
Parent PID: 1854
Child PID: 1855
Parent received: Message from child process
Parent completed.
vishalkarnati@VISHALS-MacBook-Air week12skill % █
```